A classe **BigDecimal** (presente no pacote `java.math.BigDecimal`) é usada em Java para realizar cálculos numéricos de **precisão arbitrária e exata**, evitando os problemas de arredondamento e dízimas binárias comuns dos tipos primitivos `float` e `double`. [1, 2, 3]

Como ela funciona?

Internamente, a classe BigDecimal armazena um número usando duas partes principais: [1]

1. **Valor não escalonado (Unscaled value):** Um número inteiro de tamanho arbitrário que representa os dígitos do número. [1, 2]
2. **Escala (Scale):** Um número inteiro que indica quantas casas decimais estão à direita do ponto decimal.
    - *Exemplo:* O número `3.14` é guardado com um valor não escalonado de `314` e uma escala de `2` (equivalente a 314×10^{-2}). [1, 2, 3]

Como utiliza a base decimal em vez da binária, ela consegue representar valores exatos como `0.1` sem gerar imprecisões matemáticas. [1, 2]

Principais características

- **Imutabilidade:** Os objetos do tipo BigDecimal são imutáveis. Quando você faz uma operação (como uma soma ou divisão), o objeto original não muda; um **novo** objeto é retornado com o resultado. [1, 2, 3]
- **Sem operadores diretos:** Você **não** pode usar os operadores aritméticos comuns (`+`, `-`, `*`, `/`). As operações devem ser feitas através de métodos próprios da classe. [1, 2]

Operações matemáticas básicas

Para manipular os valores, utilizam-se os seguintes métodos: [1, 2]

- **Adição:** `.add()`
- **Subtração:** `.subtract()`
- **Multiplicação:** `.multiply()`
- **Divisão:** `.divide()` (exige atenção ao arredondamento para evitar exceções caso a divisão resulte em dízima infinita) [1, 2, 3]

java

```java
import java.math.BigDecimal;

BigDecimal valor1 = new BigDecimal("0.1");
```

```
        BigDecimal valor2 = new BigDecimal("0.2");

        // Soma
        BigDecimal soma = valor1.add(valor2); // Resultado: 0.3
```

Use o código com cuidado.

Cuidados importantes

1. **Evite instanciar com `double`:** Fazer `new BigDecimal(0.1)` pode carregar a imprecisão binária original do `double`. O ideal é passar o valor como **String** (`new BigDecimal("0.1")`) ou usar métodos de fábrica como `BigDecimal.valueOf(0.1)`. [1, 2]
2. **Atenção na divisão:** Ao usar o método `.divide()`, se o resultado não tiver uma representação decimal exata e você não informar o número de casas decimais e o **modo de arredondamento** (`RoundingMode`), o Java lançará uma exceção (`ArithmeticException`). [1]
3. **Comparações:** Nunca utilize o operador `==` para comparar dois BigDecimal, pois ele compara a referência do objeto (e a escala pode ser diferente, ex: `1.0` vs `1.00`). Utilize sempre o método `.compareTo()`.